

1. Query Processor Components

Question: Explain the parts of the query processor in database system structure.

Answer: The Query Processor interprets and executes user requests. Its main parts are:

- **DDL Interpreter:** Processes Data Definition Language (DDL) statements (like CREATE or ALTER) and stores the metadata in the Data Dictionary.
 - **DML Compiler:** Translates High-level Data Manipulation Language (DML) statements (like SELECT or UPDATE) into low-level instructions.
 - **Query Optimizer:** A component of the compiler that finds the most efficient way to execute a query.
 - **Embedded DML Pre-compiler:** Handles DML statements embedded in programming languages like C++ or Java.
 - **Query Evaluation Engine:** Receives the low-level instructions and interacts with the storage manager to execute them.
-

2. Identifying Relationships

Question: "An identifying relationship between two entities is known as a strong relationship", do you agree?

Answer: No. An identifying relationship is actually the link between a Strong Entity and a Weak Entity.

- In this relationship, the child entity (Weak) is existence-dependent on the parent (Strong).
 - The relationship is called "identifying" because the parent's primary key must be used to identify the child. Calling the relationship itself "strong" is technically incorrect in ER terminology.
-

3. DBMS vs. RDBMS

Question: Explain the difference between a DBMS and an RDBMS.

Answer: * **DBMS:** Stores data as files; supports single users; has no built-in support for relationships; high data redundancy.

- **RDBMS:** Stores data in tables (relations); supports multiple concurrent users; uses Primary/Foreign keys to link data; uses **Normalization** to reduce redundancy and follows **Codd's Rules**.
-

4. Super Keys and Primary Keys

Question: "All super keys are eligible to be primary keys" - support or refute.

Answer: Refute. * A Super Key is any set of attributes that uniquely identifies a row, but it may contain redundant information (e.g., {ID, Phone_Number}).

- A **Primary Key** must be a **Candidate Key**, meaning it must be a **minimal** super key.
 - If a super key contains extra attributes, it is not "minimal" and therefore is not eligible to be a primary key.
-

5. Partial Keys and Discriminators

Question: What is a partial key or discriminator? Are descriptive attributes and mapping cardinalities related?

Answer: * **Partial Key/Discriminator:** An attribute in a Weak Entity that can only uniquely identify a record when combined with the owner entity's primary key (e.g., Room_Number in a Hotel).

- **Relationship:** Yes, they are related. In a **Many-to-Many (M:N)** relationship, descriptive attributes (like Date_of_Sale) **must** be placed in the relationship itself (the bridge table). In **One-to-Many** or **One-to-One**, these attributes are usually moved into the entity tables to simplify the schema.
-

6. Referential Integrity Scenario

Question: What happens if you insert an Order with a CustomerID that doesn't exist in the Customers table?

Answer: The RDBMS will block the insertion and throw an error. This violates the Referential Integrity Rule. This rule ensures that a Foreign Key value must always match an existing Primary Key value in the referenced table, preventing "orphan" records.

7. Concurrency Control

Question: Explain concurrency control for multiple users.

Answer: Concurrency Control ensures that multiple users can access and modify the same database simultaneously without causing data inconsistency. It prevents problems like:

- **Lost Updates:** Two users overwriting each other.
- **Dirty Reads:** Reading data that hasn't been saved (committed) yet.

It is primarily managed through Locks (Shared and Exclusive).

8. Mapping Cardinalities

Question: Define mapping cardinality, list types for binary relationships, and explain how to resolve Many-to-Many (M:N).

Answer: * **Definition:** The number of entities to which another entity can be associated via a relationship.

- **Binary Types:** One-to-One (1:1), One-to-Many (1:M), Many-to-One (M:1), and Many-to-Many (M:N).
 - **M:N Problems:** Causes high redundancy and data anomalies.
 - **Resolution:** Breakdown the M:N relationship by creating an **Associative Entity (Bridge Table)** that holds the primary keys of both participating tables as foreign keys.
-

9. SQL Filtering: WHERE vs. HAVING

Question: Difference between "where" and "having" clause.

Answer: * **WHERE:** Used to filter individual rows before grouping. It cannot use aggregate functions (\$SUM\$, \$AVG\$).

- **HAVING:** Used to filter groups **after** the GROUP BY clause. It is specifically designed to work with aggregate functions.
-

10. Identifying vs. Non-Identifying Relationships

Question: Differentiate between identifying and non-identifying relationships.

Answer: * **Identifying:** The child entity's identity is dependent on the parent. The parent's PK becomes part of the child's PK. (e.g., Building and Room).

- **Non-Identifying:** The child is independent. The parent's PK is only a Foreign Key in the child, not part of its identity. (e.g., Department and Employee).